

Informe de Implementación de PyTest y TDD en Python

INTRODUCCIÓN

En esta práctica se trabajó con la metodología TDD (Test Driven Development) utilizando Python y la herramienta PyTest.

El objetivo fue realizar pruebas automáticas sobre el proyecto desarrollado, verificando el correcto funcionamiento de la calculadora y detectando errores de manera automática.

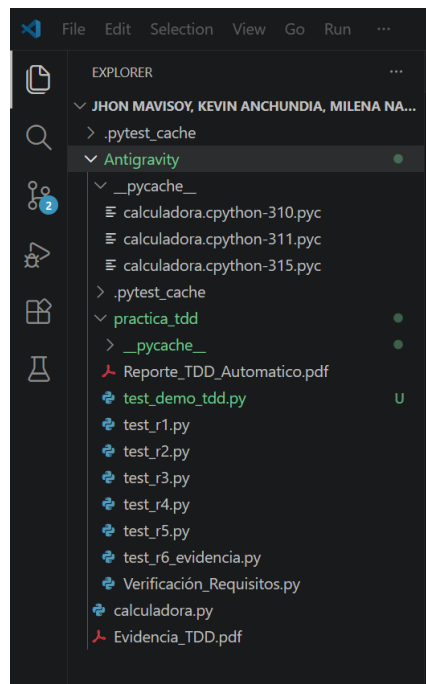
Además, se creó una nueva prueba personalizada para evidenciar el funcionamiento del testing automatizado dentro del proyecto.

DESARROLLO

1. Estructura del proyecto

El proyecto cuenta con un archivo principal llamado calculadora.py, el cual contiene la lógica del sistema.

También se incluyen varios archivos de prueba como test_r1.py, test_r2.py y otros, encargados de validar distintos requisitos mediante pruebas automáticas.



2. Creación de una nueva prueba TDD

Como parte de la práctica se creó un nuevo archivo llamado test_demo_tdd.py, utilizando PyTest para validar automáticamente un cálculo de descuento.

La prueba verifica que al aplicar un descuento del 10% a un valor de 100, el resultado obtenido sea 90.

```
← → Jhon Mavisoy, Kevin Anchundia, Milena Narvaez y Alex Moreno (1)
calculadora.py test_demo_tdd.py U X
Antigravity > practica_tdd > test_demo_tdd.py > test_descuento_basico
1 from calculadora import calcular_precio_final
2
3 def test_descuento_basico():
4     assert calcular_precio_final(100, 10) == 90
```

3. Ejecución de pruebas automáticas

Para ejecutar las pruebas del sistema se utilizó el comando:

```
python -m pytest
```

PyTest detectó automáticamente todos los archivos de prueba del proyecto, incluyendo la nueva prueba creada.

Como resultado, varias pruebas fueron aprobadas correctamente y también se detectaron errores intencionales incluidos para evidenciar el funcionamiento de la herramienta.

```
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\RESPALDOS SSD\RESPALDOS ENERO 2021 DELL\Escritorio\Septimo Semestre\Normativas de Ingenieria\Parcial 2\Jhon Mavisoy, Kevin Anchundia, Milena Narvaez y Alex Moreno (1)\Antigravity
collected 13 items

practica_tdd\test_demo_tdd.py . [ 7%]
practica_tdd\test_r1.py .. [ 23%]
practica_tdd\test_r2.py .. [ 38%]
practica_tdd\test_r3.py . [ 46%]
practica_tdd\test_r4.py . [ 53%]
practica_tdd\test_r5.py . [ 61%]
practica_tdd\test_r6_evidencia.py .FFFF [100%]

===== FAILURES =====
_____ TestR6Evidencia.test_falla_intencional_evidencia_1 _____

self = <test_r6_evidencia.TestR6Evidencia testMethod=test_falla_intencional_evidencia_1>

    def test_falla_intencional_evidencia_1(self):
        # Fallo de lógica matemática (AssertionError)
>       self.assertEqual(calcular_precio_final(100, 20), 50.0, "Forzando error: Cálculo matemáticamente incorrecto")
E       AssertionError: 80.0 != 50.0 : Forzando error: Cálculo matemáticamente incorrecto

practica_tdd\test_r6_evidencia.py:11: AssertionError
_____ TestR6Evidencia.test_falla_intencional_evidencia_2 _____

===== 4 failed, 9 passed in 0.23s =====
PS D:\RESPALDOS SSD\RESPALDOS ENERO 2021 DELL\Escritorio\Septimo Semestre\Normativas de Ingenieria\Parcial 2\Jhon Mavisoy, Kevin Anchundia, Milena Narvaez y Alex Moreno (1)\Antigravity>
```

4. Detección automática de errores

Durante la ejecución se identificaron errores como AssertionError y TypeError.

Esto demuestra cómo PyTest permite encontrar problemas de lógica, errores de datos y fallos en la ejecución de manera automática, facilitando el proceso de validación del software.

```
practica_tdd\test_r6_evidencia.py:23: TypeError
===== short test summary info =====
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_1
- AssertionError: 80.0 != 50.0 : Forzando error: Cálculo matemáticamente incorrecto
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_2
- TypeError: '<=' not supported between instances of 'str' and 'int'
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_3
- AssertionError: 50.0 != 55.0 : Forzando error: Falta agregar la propina del 10% (no requ...
FAILED practica_tdd/test_r6_evidencia.py::TestR6Evidencia::test_falla_intencional_evidencia_4
- TypeError: calcular_precio_final() missing 2 required positional arguments: 'precio_orig...
```

CONCLUSIÓN

El uso de PyTest y la metodología TDD permitió realizar pruebas automáticas sobre el proyecto desarrollado en Python.

Gracias a estas herramientas fue posible validar el funcionamiento del sistema, detectar errores automáticamente y comprobar el correcto comportamiento del código.

La práctica permitió comprender la importancia del testing automatizado en el desarrollo de software y su utilidad para mejorar la calidad de los programas.